Ders 6 Genişletilmiş Notu: Huffman Kodlama: Ağaç, Varyans, Header ve Dizi ile Kod Üretimi

Ders 6 Genişletilmiş Notu: Huffman Kodlama: Ağaç, Varyans, Header ve Dizi ile Kod Üretimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Dersin Tekrarı: Prefix Kod ve Shannon-Fano
- 2. Huffman Kodlama Yöntemi
- 3. Aynı Olasılıklardan İki Farklı Huffman Ağacı
- 4. Hangi Kod Daha İyidir: Varyans ve Buffer
- 5. Kod Uzunluğu Tahmini ve Ortalama Kod Uzunluğunun Kısa Yolu
- 6. Frekanslarla Huffman Örneği: Genişletilmiş Dizi
- 7. Ağacın Kurulması ve Kodların Okunması
- 8. Sınıf Alıştırması: 1, 1, 1, 2, 3, 5, 6, 11
- 9. Header (Başlık) Bilgisi
- 10. Ağacı Diziye Aktarmak: Düğüm Adresleri
- 11. İndislerden Kod Üretmek
- 12. Sembol Türleri
- 13. Statik, Yarı Dinamik ve Dinamik Huffman
- Örnek Sorular

1. Önceki Dersin Tekrarı: Prefix Kod ve Shannon-Fano

Hoca derse önceki haftada başlanan istatistiksel yöntemleri hatırlatarak başladı. Prefix kod "çok önemlidir", çünkü bütün istatistiksel yöntemlerde ortaya çıkan kodların hepsinin prefix olması gerekir. Bir kod prefix değilse sıkıştırılan dosya geri açılamaz, yani decode edilemez.

Ardından Shannon-Fano kodu kısaca özetlendi. Olasılıklar sıralanır; yaklaşık olarak aynı olasılık toplamına sahip olanlar ikiye bölünerek devam edilir; her bölmede bir tarafa 1, diğer tarafa 0 etiketi konur ve en sonunda kodlar elde edilir. Her şey yolunda gitmişse entropi ile ortalama mesaj uzunluğu arasındaki fark redundancy'yi (artıklığı) verir. Bu fark ne kadar küçükse o kadar iyi bir sıkıştırma yöntemi seçilmiş demektir. Fark büyükse veride hâlâ sıkıştırılacak bir şey var demektir ve sıkıştırılmış verinin üzerine ikinci bir sıkıştırma yöntemi daha uygulanabilir.

> **Ders sonrası not:** Hoca 00:14:11'de Shannon-Fano'da olasılıkların "küçükten büyüğe" sıralandığını söyledi. Önceki dersin slaytına ve örneğine göre Shannon-Fano'da semboller büyükten küçüğe (azalan sırada) sıralanır.

2. Huffman Kodlama Yöntemi

Hoca Huffman'ın "gerçekten önemli" bir kodlama yöntemi olduğunu söyledi. Günlük hayatta kullanılan sıkıştırma formatlarının büyük çoğunluğunda, ister bir resim formatı olsun ister farklı türden dosyalardan oluşan bir klasörü sıkıştıran ZIP, WinRAR gibi araçlar olsun, ara adımlardan birinde Huffman kodlaması kullanılır. Örneğin bir resim dosyasına önce quantization uygulanıp renk sayısı azaltılabilir, sonra sonuç Huffman ile kodlanabilir; ya da bir

doküman doğrudan Huffman ile kodlanabilir. Semboller tek karakterler olabileceği gibi ikili karakterler de olabilir.

Hoca Huffman'ı lisansta görmüş olanlar olabileceğini, ama bu derste özellikle **header** kısmının "altını çize çize" anlatılacağını belirtti.

Slayttaki tanım maddeleri şunlardır (ekrandan, 00:20:00):

- Veri sıkıştırmada tek başına veya bir başka yöntemin içerisinde yer alarak kullanılan bir algoritmadır.
- Shannon-Fano'ya benzer, ancak kodları ters olarak aşağıdan yukarıya doğru oluşturur.
- Huffman algoritmasında kodlama ağacı oluşturulurken iki farklı yol izlenir. Sembollerin kod değerleri farklı olsa da entropi aynı çıkar.

Shannon-Fano ile benzerliği, ikisinin de kodları bir ağaç yapısında oluşturmasıdır. Ancak Shannon-Fano çok daha basittir: olasılıkları hep yaklaşık yarıya bölerek ilerler. Huffman öyle bir algoritma değildir ve Huffman ağacının dengeli olması da gerekmez. Huffman'da ağaç yapraklardan köke doğru kurulur, ama kodlar kökten (root) yapraklara doğru okunur.

> **Ders sonrası not:** Hoca 00:17:44 ve 01:02:34'te Shannon-Fano'da kodların yapraklardan köke doğru okunduğunu söyledi. Slayttaki karşılaştırma kodların oluşturulma yönüyle ilgilidir: Shannon-Fano ağacı kökten başlayıp bölerek, Huffman ağacı yapraklardan başlayıp birleştirerek kurulur. Önceki dersteki Shannon-Fano örneğinde de kodlar kökten yaprağa okunmuştur (ilk bölmenin etiketi kodun ilk bitidir).

"İki farklı yol izlenir" ifadesinin anlamı şudur: elimizde sembollerin olasılık ya da frekans değerleri varken algoritmanın işleyişinde iki farklı Huffman ağacı oluşturulabilir. Kodlar birbirinden farklı çıkabilir; bir ağaç bir sembole `11` verirken öteki aynı sembole `10` verebilir. Ama iki ağacın entropisi hesaplandığında aynı değer çıkar. Hoca entropiyi hatırlattı: elimizdeki veriyi kodlamak istersek her sembol için ortalama kaç bit harcamamız gerektiğini verir. Entropi kaynak hakkında bilgimiz varsa, yani verinin içinde kaç farklı sembol olduğunu ve bunların dağılımını biliyorsak hesaplanabilir.

3. Aynı Olasılıklardan İki Farklı Huffman Ağacı

Hoca, kitaplardaki Huffman ağacı çiziminin (soldan sağa gelişen ağaç) kendisine "zor ve meşakkatli" geldiğini, sonraki örneklerde yukarıdan aşağıya bildiğimiz tarzda bir ağaç kullanacağını söyledi. Ama iki farklı yolun ne olduğunu göstermek için önce bu slayttaki örneği anlattı (ekrandan, 00:30:00).

Kaynağın içine girilip $a_1, \dots, a_5$ sembollerinin kaçar tane olduğu sayılmış ve olasılıklar bulunmuştur:

Sembol	a_1	a_2	a_3	a_4	a_5
Olasılık	0,4	0,2	0,2	0,1	0,1

Olasılıklar büyükten küçüğe ya da küçükten büyüğe sıralanabilir; bu gösterimde bunun önemi yoktur. Küçük olan taraftan başlanır ve en küçük iki olasılık toplanır.

3.1 Birinci ağaç: eşit değerin önüne koymak

- $a_4 + a_5 = 0{,}1 + 0{,}1 = 0{,}2$ (a_{45}). Bu yeni 0,2 sıralamayı bozmayacak şekilde diziye yerleştirilmelidir. Dizide zaten iki tane 0,2 vardır; yeni 0,2 ya onların önüne ya da arkasına konabilir. "İki farklı ağaç oluşturulur"un anlamı budur. Birinci ağaçta önüne konur.
- Yeni 0,2 ile a_3'ün 0,2'si toplanır: 0,4 (a_{345}). Dizide zaten bir 0,4 vardır. Algoritmada bir kez önüne konduysa, aynı değerde bir şey geldiğinde hep önüne konmalıdır; 0,4 de mevcut 0,4'ün önüne gider.
- a_2'nin 0,2'si ile 0,4 toplanır: 0,6 (a_{2345}).
- 0,6 ile a_1'in 0,4'ü toplanır: 1,0 (a_{12345}).

Kodlar 1 ve 0'lardan oluşur; dallara etiket verilir: alt dallara 0, üst dallara 1. Kodlar kökten yapraklara doğru okunur:

- a_1: `0`
- a_2: `10`
- a_3: `111`
- a_4: `1101`
- a_5: `1100`

Ortalama mesaj uzunluğu, her sembolün olasılığı ile o sembolü kodlamak için gereken bit sayısının çarpımlarının toplamıdır (ekrandan, 00:30:00):

$$S = 0{,}4 \cdot 1 + 0{,}2 \cdot 2 + 0{,}2 \cdot 3 + 0{,}1 \cdot 4 + 0{,}1 \cdot 4 = 2{,}2 \text{ bit/sembol}$$

Entropi hesaplanıp ortalama mesaj uzunluğundan çıkarılınca redundancy bulunur.

> **Ders sonrası not:** Hoca 00:28:03'te a_4'ün olasılığını 0,4 diye söyledi; slayta göre a_4'ün olasılığı 0,1'dir. Hesap slayttaki gibi doğrudur.

3.2 İkinci ağaç: eşit değerin arkasına koymak

Aynı olasılıklar (0,4; 0,2; 0,2; 0,1; 0,1) için bu kez eşit değerlerin arkasına konur:

- $0{,}1 + 0{,}1 = 0{,}2$ (a_{45}); 0,2'lerin bittiği yere konur.
- Sıradaki iki değer a_2 ve a_3'tür: $0{,}2 + 0{,}2 = 0{,}4$ (a_{23}); mevcut 0,4'ün arkasına konur.
- 0,2 (a_{45}) +0,4 (a_1) = 0,6 (a_{145}).
- 0,4 (a_{23}) +0,6 = 1,0.

Ekrandaki ağaçtan okunan kodlar: a_1 = `11`, a_2 = `01`, a_3 = `00`, a_4 = `101`, a_5 = `100` (ekrandan, 00:30:00 ve 00:40:00). Ağaç üç seviyelidir; birinci ağaç dört seviyeliydi.

Kod uzunlukları birinci ağaçta 1, 2, 3, 4, 4; ikinci ağaçta 2, 2, 2, 3, 3'tür. İkinci ağaçta bir bitlik kod yoktur, en uzun kod üç bittir. Buna rağmen ortalama mesaj uzunluğu aynıdır:

$$S_2 = 0{,}4 \cdot 2 + 0{,}2 \cdot 2 + 0{,}2 \cdot 2 + 0{,}1 \cdot 3 + 0{,}1 \cdot 3 = 2{,}2 \text{ bit/sembol}$$

İki ağacın entropileri ve dolayısıyla redundancy'leri de birbirine eşittir. İki farklı ağaç, farklı kodlar, ama aynı performans.

> **Ek bilgi (derste anlatılmadı):** Bu dağılımın entropisi python ile $H = -\sum P_i \log_2 P_i \approx 2{,}12$ bit çıkar. Dolayısıyla iki ağacın redundancy'si de $R = 2{,}2 - 2{,}12 \approx 0{,}08$ bittir.

Peki hangisi seçilmeli? Kendi bilgisayarında, kendi diskinde bir dosyayı sıkıştırıyorsan (statik ortam) hangi ağacı kullandığının önemi yoktur. Ama bir A noktasından B noktasına veri iletirken (dinamik Huffman) ikinci kodu seçmek daha idealdir, çünkü bu kodun varyansı daha düşüktür. Hoca, eşit değerleri arkaya koyarak yerleştirmenin bu örnekte küçük varyanslı ağacı verdiğini, önüne koymanın varyansı büyütebildiğini belirtti.

4. Hangi Kod Daha İyidir: Varyans ve Buffer

4.1 Varyans nedir?

Varyans, bir dizinin elemanları arasındaki değişimi ölçer. Hoca iki dizi örneği verdi: `2, 1, 2, 3, 1, 2` dizisinde elemanlar arasındaki değişim küçüktür, varyans küçüktür; `1, 25, 17, 55, 4` dizisinde elemanlar arasındaki fark büyüktür, varyans büyüktür. Varyans standart sapmanın karesidir; standart sapma her elemanın ortalamaya ne kadar yakın olduğunu verir.

Kod için varyans, her sembolün kod uzunluğunun ortalama kod uzunluğundan farkının karesi, olasılıkla ağırlıklandırılarak toplanır:

$$\sigma^2 = \sum_i P_i \, (l_i - S)^2$$

4.2 İki ağacın varyansı

Slayttaki hesap (ekrandan, 00:40:00 ve 00:44:20):

Birinci ağaç (uzunluklar 1, 2, 3, 4, 4):

$$0{,}4(1-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(3-2{,}2)^2 + 0{,}1(4-2{,}2)^2 + 0{,}1(4-2{,}2)^2 = 1{,}36$$

İkinci ağaç (uzunluklar 2, 2, 2, 3, 3):

$$0{,}4(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}2(2-2{,}2)^2 + 0{,}1(3-2{,}2)^2 + 0{,}1(3-2{,}2)^2 = 0{,}16$$

Ara adımlar (ara adım eklendi): birinci ağaçta $0{,}4 \cdot 1{,}44 + 0{,}2 \cdot 0{,}04 + 0{,}2 \cdot 0{,}64 + 2 \cdot 0{,}1 \cdot 3{,}24 = 0{,}576 + 0{,}008 + 0{,}128 + 0{,}648 = 1{,}36$; ikinci ağaçta $0{,}4 \cdot 0{,}04 + 2 \cdot 0{,}2 \cdot 0{,}04 + 2 \cdot 0{,}1 \cdot 0{,}64 = 0{,}016 + 0{,}016 + 0{,}128 = 0{,}16$. İki sonuç da python ile doğrulandı.

İkinci ağacın varyansı daha küçüktür: kod uzunlukları `2, 2, 2, 3, 3` gibi birbirine yakındır; birinci ağaçta ise `1, 2, 3, 4, 4` gibi dağılmıştır. Hoca bu örnekte farkın çok büyük görünmeyebileceğini, ama gerçek dünya uygulamalarında daha belirgin olacağını söyledi.

> **Ders sonrası not:** Hoca 00:36:40'ta varyans hesabını anlatırken ortalama mesaj uzunluğunu bir kez "0,2" diye söyledi; doğrusu 2,2'dir ve slayttaki hesapta da 2,2 kullanılır.

4.3 Neden küçük varyans?

Slayttaki maddeler (ekrandan, 00:40:00):

- Kodlayıcı, sıkıştırılmış veriyi bir dosyaya yazacak ise, kodun varyansının küçük veya büyük olmasının bir önemi yoktur.
- Kodlayıcı, sıkıştırılmış veriyi iletişim hatlarından gönderecek ise, küçük varyanslı kod seçilmelidir.
- İletişim hattı üzerinden bilginin gönderilme hızı ile kodlayıcının kodu üretme hızı aynı değilse beklemeyi önlemek için buffer alanı kullanılır.
- Kodun varyansı büyük ise buffer alanı büyük seçilmelidir. Varyansın sıfıra yakın olduğu durumlarda buffer alanı küçük seçilebilir.

Hoca bunu tahtaya çizdiği bir A-B hattıyla anlattı. Küçük varyanslı kodda a_1 2 bitlik gelir, B açar; arkasından a_3 2 bitlik gelir, B onu da açar. Ama varyansı büyük bir kod olsaydı, örneğin a_1 5 bit, a_3 1 bit olsaydı: 5 bitlik kod gönderilir; B bu kodu çözene kadar arkadan hızla 1 bitlik kodlar gelir. B hâlâ 5 bitlik (ya da 20 bitlik) kodu çözerken durmadan daha kısa kodlu semboller gönderilir ve bir birikme olur. Hattın iletim hızına çözücünün decode etme hızı yetişemez.

O zaman bilgilerin kaybolmaması için bir buffer alanı açmak gerekir: uzun kod çözülürken geri kalan kodlar bu buffer alanında sırasını bekler. Bu alan bellekten (memory) ayrılır ve bir bekleme söz konusu olur. Beklemeyi azaltmak için kodların bit uzunluklarının birbirine yakın olması, yani varyansın küçük olması gerekir.

> **Hocanın vurgusu:** Varyansı küçük kodu seçmezsen "büyük bir hata olmaz, ama her işin bir raconu var." Statik ortamda hangi ağacı kullandığının önemi yoktur; iletimde küçük varyanslı kodu seçmek işini kolaylaştırır.

5. Kod Uzunluğu Tahmini ve Ortalama Kod Uzunluğunun Kısa Yolu

5.1 $\lceil -\log_2 P_i \rceil$ ile tahmin

Bir sembolün kod uzunluğu tahmin edilebilir. Slayttaki ifade (ekrandan, 00:44:20):

- a_i sembolünün kod uzunluğu $\lceil -\log_2 P_i \rceil$'ye eşit veya daha küçüktür.
- $a_i \to 0{,}4$: $\lceil -\log_2 0{,}4 \rceil = 2$ bit (2 veya 1 bit ile kodlanır).

Hoca tavan fonksiyonunu (“yukarıya doğru kıvrımlı olan şey”) önceki dersten hatırlattı: sonuç kendisinden büyük ya da eşit en küçük tam sayıya eşitlenir. $-\log_2 0{,}4 \approx 1{,}32$ olduğundan sonuç 2’dir; yani olasılığı 0,4 olan sembol en fazla 2 bitle kodlanır. Hoca, olasılığı 0,4 olan bir sembol 3 bitle kodlanmışsa kodda mutlaka bir hata yapıldığını, bir yerde yanlış işlem yapılıp yanlış uzunlukta kodlandığını söyledi.

> **Ders sonrası not:** Bu üst sınır Huffman kodlarında her sembol için garanti değildir. Derste kurulan Bölüm 6-7’deki örnekte `m` sembolünün olasılığı $4/60 \approx 0{,}067$’dir; $\lceil -\log_2 0{,}067 \rceil = 4$ olduğu hâlde Huffman kodu `00010` 5 bittir (python ile kontrol edildi). Huffman kodu ortalama uzunluk bakımından en iyidir, ama tek tek sembollerde bu sınır aşılabilir. Bu yüzden sınırın aşılması tek başına hata kanıtı sayılmamalıdır; yine de büyük sapmalar bir hatanın işareti olabilir.

5.2 Ortalama kod uzunluğu = ara düğümlerin toplamı

Hoca ilk slayttaki ağaca döndü (ekrandan, 00:46:00). Ortalama kod uzunluğu her sembolün olasılık değeri ile kod bit uzunluğunun çarpımlarının toplamı olarak hesaplanıyordu. Ama aynı değer, Huffman kodlama ağacının **ara düğüm** olasılık değerleri toplanarak da bulunabilir. Birinci ağaçta ara düğümler 0,2 (a_{45}), 0,4 (a_{345}), 0,6 (a_{2345}) ve 1,0’dır (kök). Ekranda birikimli toplamlar yazılmıştır: $0{,}2 + 0{,}4 = 0{,}6$; $0{,}6 + 0{,}6 = 1{,}2$; $1{,}2 + 1{,}0 = 2{,}2$. Sonuç, formülle hesaplanan $S = 2{,}2$ ile aynıdır. Böylece her sembolün olasılığını bit sayısıyla çarpıp toplamakla uğraşmaya gerek kalmaz.

> **Ek bilgi (derste anlatılmadı):** Bunun nedeni, her sembolün olasılığının kökten kendisine kadar geçtiği her ara düğümün değerine bir kez katkı yapmasıdır. Kod uzunluğu l_i olan bir sembol tam l_i ara düğümün içinde sayılır; bu yüzden ara düğümlerin toplamı $\sum P_i l_i$’ye eşittir.

6. Frekanslarla Huffman Örneği: Genişletilmiş Dizi

Hoca bundan sonra, kendisine “daha yakın bulduğu” ve öğrencilere de önerdiği, yukarıdan aşağıya ağaçla çalışılan yöntemi bir örnek üzerinde anlattı (ekrandan, 00:50:00).

> Elimizde a, e, k, l, m ve z sembollerinden sırası ile 25, 14, 8, 7, 4 ve 2 adet olsun. Her bir sembol için Huffman kod değerini bulalım.

Kitaplar bazen frekans değerlerini, bazen olasılık değerlerini kullanır. Hoca olasılıkların “sıfır virgül bir şeyler” olduğu için yazması ve slayta koyması zor geldiğinden frekansları kullandığını söyledi. İkisi de doğrudur.

6.1 Adım 1: sıralama

Sembollerin kullanım sıklıkları (ya da olasılıkları) küçükten büyüğe sıralanır:

```
2   4   7   8   14   25
```

6.2 Adım 2: toplamları diziye yerleştirme

Slayttaki kural: sıklık değerleri, soldan başlayarak sıradaki iki sıklık değerinin toplamlarının sıralamayı bozmayacak şekilde diziye yerleştirilmesi ile devam edilir.

- $2 + 4 = 6$. Sıralamayı bozmadan 4 ile 7’nin arasına yerleştirilir.
- Sıradaki ikili 6 ile 7’dir: $6 + 7 = 13$. 8 ile 14 arasına yerleştirilir.
- Sıradaki ikili 8 ile 13: $8 + 13 = 21$. 14 ile 25 arasına yerleştirilir.
- Henüz toplama girmemiş sıradaki iki değer 14 ile 21: $14 + 21 = 35$. 25’ten büyük olduğu için 25’ten sonra gelir.
- $25 + 35 = 60$. 35’ten sonraya yerleşir.

60’ı toplayabilecek başka değer kalmamıştır; bu, gelinebilecek yere kadar gelindiği, işlemin bittiği anlamına gelir. Genişletilmiş dizi (ekrandan, 00:54:30):

```
2   4   -6   7   8   -13   14   -21   25   -35   -60
```

> **Hocanın vurgusu:** Kod yazarken ya da sınavda sık yapılan hata: 2 ile 4'ü toplayıp yazmak, sonra 4 ile 6'yı, sonra 6 ile 7'yi toplamak. "Yok öyle." Toplama girmiş bir değer bir daha toplamaya sokulmaz.

6.3 Eksi işaretinin anlamı

Hoca tahtada kırmızı ve siyah kalemle yazdığı için gözle hangi değerin gerçek frekans, hangisinin toplam olduğu görülebilir: siyahlar sembollerin gerçek frekansları, kırmızılar frekansların toplamından oluşan değerlerdir. Ama kod yazarken bilgisayara kırmızı rengi anlatmanın yolu yoktur. Bu yüzden toplamların önüne eksi konur: gördüğün sayı eksi ise bir şeylerin toplamıdır (ara toplam); pozitifse bir sembolün frekansıdır. Eksi yalnız bu farkı bilgisayara anlatmak içindir. Toplama yapılırken mutlak değerler kullanılır: -6 ile 7 toplanınca $+1$ değil, 13 yazılır ve önüne yine eksi konur.

Bu örnekte toplam hiçbir zaman mevcut bir gerçek frekansa eşit çıkmadı. Örneğin toplam 13 çıktığında dizide siyah bir 13 olsaydı, yeni 13 onun önüne mi arkasına mı konacaktı? Hoca bunun kişiye kaldığını söyledi: dinamik bir şeyle uğraşılmıyorsa ister önüne ister arkasına yazılır. Ama algoritmada bir kez sola yazıldıysa aynı değer geldiğinde hep sola, sağa yazıldıysa hep sağa konmalıdır.

6.4 Genişletilmiş dizinin boyu

Slayttaki kural: n tane sembolümüz varsa, genişletilmiş dizimiz $2n-1$ adet olacaktır (ekrandan, 00:54:30). Burada $n = 6$, $2 \cdot 6 - 1 = 11$; dizide gerçekten 11 eleman vardır. Bu sayı aynı zamanda durma noktasını da verir.

Bazı kaynaklar n sembol için genişletilmiş dizinin $2n-2$ adet olduğunu söyler. Bu durumda 10 eleman olur, yani en sondaki -60 alınmaz; dizi -35, 25 diye bitirilir ve kök bilgisi kullanılmaz. Hoca bunu önermedi: $2n - 1$ ile hesaplanmalı, sonda her zaman tek bir değer kalmalı ve o değer kök kabul edilmelidir.

7. Ağacın Kurulması ve Kodların Okunması

7.1 Adım 3: ağaç

Ağaç oluşturulurken genişletilmiş dizi sondan başa doğru okunur. En sonda tek başına kalan değer (-60) başka bir şeye bağlanamaz; bu köktür (root). Huffman ağacı bir ikili ağaçtır (binary tree): bir düğümden en fazla iki dal çıkar ya da hiç çıkmaz. Kök bir ara toplam olduğu için altından iki dal çıkacaktır.

Kökten sonra okunan iki değer onun çocuklarıdır: **ilk okunan sola, ikinci okunan sağa** yazılır.

- -60'ın altı: -35 (sol) ve 25 (sağ). $35 + 25 = 60$. 25 pozitif (siyah), gerçek bir değerdir; yapraktır. -35 negatif (kırmızı), bir şeylerin toplamıdır; ağaç onun altından devam eder.
- -35'in altı: -21 (sol) ve 14 (sağ). $21 + 14 = 35$. 14 yapraktır, -21 ara düğümdür.
- -21'in altı: -13 (sol) ve 8 (sağ). $13 + 8 = 21$.
- -13'ün altı: 7 (sol) ve -6 (sağ). $7 + 6 = 13$.
- -6'nın altı: 4 (sol) ve 2 (sağ). $4 + 2 = 6$.

Yapraklara semboller yerleştirilir: $25 \to$ `a`, $14 \to$ `e`, $8 \to$ `k`, $7 \to$ `l`, $4 \to$ `m`, $2 \to$ `z` (ekrandan, 01:00:00).

Bu ağaç ikili ağaçtır; dengeli olup olmaması önemli değildir. Ağacın yüksekliği (level sayısı) önemlidir: kökten yaprağa giden en uzun yolda 5 dal vardır; demek ki verilebilecek en uzun kod 5 bittir. Kökten gidilebilen en kısa yaprak 1 dal uzaklıktadır; en kısa kod 1 bittir. Bu ağaçta 1 ile 5 bit arasında kodlar olacaktır.

7.2 Adım 4: dallara etiket

Dallara etiket verilirken yine tek bir kural tutarlı uygulanmalıdır: ya bütün sol dallara 0, sağ dallara 1; ya da bütün sol dallara 1, sağ dallara 0. "Kafama göre burada sola 0, şurada sola 1 verdim, yok." Hoca her zamanki alışkanlığıyla sola 0, sağa 1 verdi.

7.3 Adım 5: kodların okunması

Slayttaki başlık: Huffman kodlama ağacında kodlar aşağıdan yukarı doğru oluşturulur, yukarıdan aşağı doğru da okunur (ekrandan, 01:05:40). Kökten yaprağa kadar geçilen dalların etiketleri sırayla yazılır:

Sembol	Frekans	Kod
a	25	`1`
e	14	`01`
k	8	`001`
l	7	`0000`
m	4	`00010`
z	2	`00011`

Bunlar prefix kod mudur? `1` ile başlayan başka kod yok; `01` ile başlayan başka kod yok; `001` ile başlayan başka kod yok; `0000` ile başlayan başka kod yok; `0001` ile başlayan iki kod var ama ikisi de 5 bit ve biri diğerinin ön eki değil. Öyleyse bu prefix koddur.

7.4 Kodlar ters okunursa

Hoca "boş bulunup" kodları yukarıdan aşağı değil aşağıdan yukarıya (yapraktan köke) okuduğumuzu düşünelim, dedi. Bu durumda kodlar şöyle çıkar (ekrandan, 01:05:40): `a = 1`, `e = 10`, `k = 100`, `l = 0000`, `m = 01000`, `z = 11000`.

Bu prefix kod değildir: `1` ile başlayan birden çok kod vardır (`1`, `10`, `100`, `11000`). Alıcı bir `1` okuduğunda bunun `a` mı olduğunu, yoksa ikinci biti alıp `e` mi, üçüncü bite bakıp `k` mı olduğunu bilemez; karar veremez. Kod belki açılır ama bambaşka karakterlerden oluşan bir çıktıyla karşılaşılır.

> **Hocanın vurgusu:** Prefix olması lazım; prefix olabilmesi için de kodların yukarıdan aşağıya okunması lazım. Kodlar aşağıdan yukarıya oluşturuluyor, ama yukarıdan aşağıya okunuyor.

8. Sınıf Alıştırması: 1, 1, 1, 2, 3, 5, 6, 11

Ara vermeden önce hoca ne kadar anlaşıldığını görmek için bir alıştırma yaptırdı: sembollerin frekans değerleri 1, 1, 1, 2, 3, 5, 6 ve 11 olarak verilsin; Huffman ağacını oluşturun (ekrandan, 01:10:00 ve 01:17:45).

8.1 Dizinin genişletilmesi

Sembol sayısı $n = 8$'dir; genişletilmiş dizi $2n - 1 = 15$ eleman olacaktır.

> **Ders sonrası not:** Hoca 01:08:26'da önce "16, 17 tane eleman" dedi; bir öğrencinin itirazı üzerine $16 - 1 = 15$ diye düzeltti.

Toplamlar:

- $1 + 1 = 2$. Dizide zaten bir 2 vardır. Statik çalışıldığı için yeni 2'nin sola ya da sağa konmasının önemi yoktur; sağına konur.
- Sıradaki ikili: kalan 1 ve gerçek 2: $1 + 2 = 3$. Dizide 3 vardır; yeni 3 yerleştirilir.
- Sıradaki ikili: ara toplam 2 ve gerçek 3: $2 + 3 = 5$. Dizide 5 vardır; yeni 5 yerleştirilir.
- $3 + 5 = 8$ (ara toplam 3 ve gerçek 5).
- $5 + 6 = 11$ (ara toplam 5 ve 6). Dizide 11 vardır; sağına konur.
- $8 + 11 = 19$ (ara toplam 8 ve gerçek 11).
- $11 + 19 = 30$. Tek değer kalmıştır; dizi 15 elemana ulaşmıştır, işlem biter.

> **Ders sonrası not:** Hoca 01:10:08'de yeni 3'ü mevcut 3'ün "soluna" yerleştirdiğini söyledi ("her zaman varsa soluna yerleştiriyordum"). Ekrandaki ağaçta 5 ara düğümünün çocukları gerçek 3 (yaprak) ve ara toplam 2'dir; bu ancak yeni 3 gerçek 3'ün sağına konduğunda oluşur. Bu alıştırmada eşit toplamlar tutarlı olarak hep sağa

konmuştur (2, 3, 5 ve 11 için). Önemli olan, hangi taraf seçildiyse hep o tarafın kullanılmasıdır.

Soru: Ara toplamları neden toplanan iki elemanın hemen yanına değil de daha ileriye yazıyoruz? **Cevap:** Çünkü ara toplamlar mutlaka sıralamayı bozmayacak şekilde orijinal dizinin içine yerleştirilmelidir. Bunun anlamı ağaç kurulurken ortaya çıkar: dizi sondan okunurken her ara düğümün hemen arkasından gelen ikili, onun çocuklarıdır. Örneğin 30'u veren 19 ile 11'dir; 19'u veren 11 ile 8'dir. Toplam, toplandığı yere değil belirli bir yere yazıldığı için bu ilişki korunur.

8.2 Ağaç ve kodlar

30 köktür. Ekrandaki ağaç (ekrandan, 01:17:45):

```
30
├─0─ 19
│    ├─0─ 11 (yaprak)
│    └─1─ 8
│         ├─0─ 5 (yaprak)
│         └─1─ 3
│              ├─0─ 2 (yaprak)
│              └─1─ 1 (yaprak)
└─1─ 11
     ├─0─ 6 (yaprak)
     └─1─ 5
          ├─0─ 3 (yaprak)
          └─1─ 2
               ├─0─ 1 (yaprak)
               └─1─ 1 (yaprak)
```

Sol dallara 0, sağ dallara 1 verildi. Kodlar yukarıdan aşağıya okunur. Sembollerin ne olduğu bilinmediği için frekanslarıyla anılır:

Frekans	Kod
11	`00`
5	`010`
2	`0110`
1	`0111`
6	`10`
3	`110`
1	`1110`
1	`1111`

Bunların her biri prefix koddur.

> **Ek bilgi (derste anlatılmadı):** Python ile: ağırlıklı toplam $11 \cdot 2 + 6 \cdot 2 + 5 \cdot 3 + 3 \cdot 3 + 2 \cdot 4 + 3 \cdot (1 \cdot 4) = 78$ bit, sembol başına $78/30 = 2{,}6$ bittir. Bölüm 5.2'deki kısa yolla ara düğümlerin toplamı da $30 + 19 + 11 + 8 + 5 + 3 + 2 = 78$ çıkar.

Hoca ara vermeden önce şunu vurguladı: hazır bir Huffman kodu bulunup kullanılabilir; ama öyle bir problemle karşılaşılabilir ki hazır kodun içinde bir şeyleri değiştirmek gerekir. Nereye müdahale edileceğini bilmek için algoritmanın nasıl çalıştığını bilmek gerekir.

9. Header (Başlık) Bilgisi

9.1 Neden header?

Bir input alınır, bir takım işlemlerden geçirilip sıkıştırılır. Karşı tarafın bu sıkıştırılmış veriyi açabilmesi için başına **header** denen bir bilgi eklenir; karşı taraf encode edilmiş dosyayı decode etmek için gerekli bilgileri buradan çeker. Hoca bitmap dosyalarını hatırlattı: bitmap'in

de bir header'ı vardır ve orada dosyanın boyutu, satır ve sütun sayısı, resmin siyah-beyaz mı, kaç bitlik grayscale mi, color table'lı mı, true color mı olduğu gibi bütün bilgiler bulunur.

Bir doküman sıkıştırılıp karşıya gönderildiğinde karşı tarafın en azından hangi sembole hangi kodun verildiğini bilmesi gerekir ki `1` gördüğünde "bu A'ymış", `10` gördüğünde "bu B'ymiş" diyebilsin. Dokümanda kaç farklı sembol olduğunu da bilmesi gerekir.

9.2 Seçenekler ve sorunları

Hoca öğrencilere sordu (derste sözlü, 01:40:31). Üç sembol olsun: A'dan 175, B'den 512, C'den 1 tane. Diyelim ki C'ye `0`, B'ye `10`, A'ya `110` kodu verildi. Bu bilgilerle Huffman ağacı oluşturulup veri sıkıştırıldı ve karşıya gönderildi. Karşı tarafın açabilmesi için neye ihtiyaç var?

Soru: Karşı tarafa harflerin hangi koda karşılık geldiği bilgisi mi gönderilmeli? **Cevap:** Hoca bu fikri adım adım inceledi:

- **Yalnız semboller ve arkasından kodlar:** Header'a A, B, C yazılır, arkasına kodlar arka arkaya konur. Alıcı A'nın kodunun `11` mi, `110` mu, `1101` mi olduğunu bilemez. Kodlar sabit uzunluklu olsaydı "her kod 5 bitlik, sırayla 5'er bit oku" demek yeterdi; ama kodlar değişken uzunlukludur.
- **Kodlar ve kod uzunlukları:** A'nın arkasından 3 gönderilir (3 bit oku), B'nin arkasından 2. Ama bu uzunlukların her biri bayt olarak yer tutar; 800 farklı sembol varsa ekstradan 800 bayt gerekir.
- **Semboller ve frekansları:** "Kodu niye gönderiyoruz, karşı taraf aynı ağacı kendisi kursun" denebilir: ABC ve arkasından 175, 512, 1 gönderilir. Ama bu sayıların bit uzunlukları farklıdır; 1 değeri 1 bitle yazılabilirken diğerleri çok daha fazla bit ister. Ya her birine 1 bayt ayrılır (1 değeri için bile), ya da bazı değerler bir bayta sığmayacağı için 2 bayt ayrılır; o zaman da hangisinin 1, hangisinin 2 baytla okunacağı bilgisini eklemek gerekir. 1500 farklı sembol olursa 1500 sembol ve 1500 frekans gönderilir.

  Ders sonrası not: Hoca 01:43:46'da 512'nin 8 bitle yazılabildiğini söyledi. $512 = 2^9$ olduğundan 10 bit gerekir ve tek bayta sığmaz; 175 ise 8 bittir. Bu, hocanın hemen ardından yaptığı "bazı değerler bir bayta sığmaz" uyarısını destekler.

Ayrıca frekanslar gönderildiğinde alıcı, gönderenin yaptığı uzun süreci baştan yapmak zorunda kalır: frekansları sıralamak (sort), diziyi genişletmek (aralara yerleştirip kaydırmak, merge), kodları oluşturmak.

> **Hocanın vurgusu:** Header'ın normal sıkıştırılmış veriden büyük olduğu durumlarla karşılaşılabilir. O zaman sıkıştırmanın anlamı kalmaz, orijinali gönderilir. Amaç sıkıştırmaksa header'ın çok büyük olması "abesle iştigaldir"; header da küçültülmelidir.

9.3 Çözüm: ağacın kendisini göndermek

Slayttaki soru: "Başlık içerisinde ağaç bilgisini gönderebilir miyiz? Nasıl?" (ekrandan, 01:40:00). Karşı taraf kodları çıkarmakla uğraşmasın; ağacın tamamı gönderilsin. Ağacın fotoğrafı gönderilemeyeceğine göre ağaç bir notasyonla okunur. Hoca veri yapıları, derleyici tasarımı ya da ayrık matematikten bilinen ağaç dolaşma notasyonlarını (left-node-right, left-right-node, node-left-right; infix, postfix, prefix) hatırlattı.

Slayttaki yöntem: ağacımızı **Left Node Right** notasyonuna göre okuyalım (ekrandan, 01:50:00 ve 02:00:00). Kökten başlanır; önce sola gidilir ve geçilen dalın etiketi yazılır:

- Kökten sola: `0`. -35'ten sola: `0`. -21'den sola: `0`. -13'ten sola: `0`. Artık sola gidilemiyor; bu bir yaprak, sembolü `l`.
- Left'ten sonra node'a, oradan right'a: -13'ün sağ dalı `1`. Yeni düğüm (-6); sola: `0`. Sola gidilemiyor, yaprak: `m`.
- Node'a çık, right'a in: `1`. Yaprak: `z`. -6'nın sağı ve solu bitti; -13'ün de sağı ve solu bitti.
- -21'in sağı: `1`. Yaprak: `k`.
- -35'in sağı: `1`. Yaprak: `e`.
- Kökün sağı: `1`. Yaprak: `a`.

Ekrandaki sonuç (ekrandan, 02:00:00):

```
Dal etiketleri : 0000101111
Semboller      : l m z k e a
```

9.4 Header'ın boyutu

Hoca header'ı kutu kutu çizdi (ekrandan, 01:54:05). Sembol sayısı (6) için 1 bayt, `l, m, z, k, e, a` sembollerinin her biri için 1 bayt: şimdiye kadar 7 bayt. Arkasından dal etiketleri gelir. Bunlar birer bittir, her biri için bayt harcanmaz: ilk 8 bit bir bayta, kalan 2 bit ikinci bir bayta yazılır; bu baytın geri kalanı sıfırla doldurulur (padding) ya da sıkıştırılmış verinin devamı konabilir, ama gerek yoktur. Toplam:

$$1 + 6 + 2 = 9 \text{ bayt}$$

9 baytlık header ile ne frekans değerleri, ne kodların kendisi, ne de hangi kodun kaç bit olduğu gönderilir. Header'ın arkasına sıkıştırılmış veri eklenir ve karşıya gönderilir. Gerekirse header'a ek bilgiler de konabilir.

9.5 Alıcının ağacı yeniden kurması

Karşı taraf header'ı aldığında ağacı gönderenin oluşturduğu gibi kurar (ekrandan, 02:00:00):

- `0` geldi: soldan bir dal var demektir, sola gidilir. Arkasından üç `0` daha: üç kez daha sola gidilir.
- Arkasından `0` değil `1` geldi: bu güzergâhta gidilecek yer bitmiştir; bulunulan yer yapraktır. Sıradaki sembol (`l`) oraya konur. Gelen `1`, bir üst düğüme çıkılıp sağ dala gelindiğini gösterir.
- Arkasından `0`: buradan sola bir dal gidiyor. Sonra `1`: burası bitti, yaprak; `m` çekilir. `1` ile sağ dal: arkasından `0` gelmeden yine `1` geldiği için bu da yapraktır; `z` alınır.
- Bu düğümün sağı ve solu var; bir üst düğümün sağına çıkılır (`1`). Arkasından yine `1` gelir; alt dala devam edilseydi önce `0` gelmesi gerekirdi, demek ki burası yaprak: `k`.
- Bir üst düğümün sağı (`1`): yaprak, `e`. Kökün sağı (`1`): yaprak, `a`.

Altı sembol yerleşince header'ın söylediği 6 sembol tamamlanmış, ağaç kurulmuş olur. Hoca kuralı şöyle özetledi: `0` sol demek; dört sol gelindiyse bir eleman çek. `1` gelirse yana geçilir; arkasından gelen `0` ise devam edilir. Arka arkaya iki `1` gelirse bir üst seviyeye geçilmiştir, o düğümün sıfırı bitmiştir.

Sonra gelen veri çözülür: örneğin `0` gelir; `0` olan bir kod yoktur, ikinci bit alınır. `00` diye de kod yoktur; üçüncü bit `1`: `001` = `k`. Çıktıya `k` yazılır ve sonraki bitlerle aynı işlem sürer, ta ki orijinal input oluşana kadar.

10. Ağacı Diziye Aktarmak: Düğüm Adresleri

Ağacı kâğıt üzerinde çizmek kolaydır; ama kod yazılırken bu ağaç nasıl oluşturulacak? Hoca bunun "daha da basit" olduğunu söyledi. Diziler ya da matrisler üzerinde işlem yapıldığı için ağacın her elemanını dizinin belirli bir indisine yerleştirecek bir etiketleme gerekir.

Slayttaki kural (ekrandan, 02:10:00): ağaç üzerindeki her düğüm noktasının adreslerini yazalım. Kök düğümün adresi 0 ile başlasın (yeşil ile yazılmış sayılar).

- Sol taraftaki düğüm kendisinin $\cdot 2 + 1$'i olsun.
- Sağ taraftaki düğüm de $\cdot 2 + 2$ olsun.

Örnek ağaçta: kök (-60) 0; solu $0 \cdot 2 + 1 = 1$ (-35), sağı $0 \cdot 2 + 2 = 2$ (25). -35'in solu $1 \cdot 2 + 1 = 3$ (-21), sağı $1 \cdot 2 + 2 = 4$ (14). -21'in solu 7 (-13), sağı 8 (8). -13'ün solu 15 (7), sağı 16 (-6). -6'nın solu $16 \cdot 2 + 1 = 33$ (4), sağı $16 \cdot 2 + 2 = 34$ (2).

Dizi bu indislere göre doldurulur, elemanlar sırayla yerleşmez: 0. gözde -60, 1'de -35, 2'de 25, 3'te -21, 4'te 14, 5 ve 6 boş, 7'de -13, 8'de 8, 9-14 boş, 15'te 7, 16'da -6, 33'te 4, 34'te 2. Böylece ağaç olduğu gibi diziye yerleşir ve diziye bakarak hangi yaprağın ağacın neresinde olduğu bulunur.

Bu adresler kafadan verilmiş değildir. Ağaç ikili ağaç olduğu için level 0'da $2^0 = 1$, level 1'de $2^1 = 2$, level 2'de $2^2 = 4$, level 3'te $2^3 = 8$ göz vardır ve gözler sıfırdan başlayarak sırayla numaralanır. Örneğin level 1'de 1-2, level 2'de 3-6, level 3'te 7-14, level 4'te 15-30, level 5'te 31-62 numaralı gözler bulunur. Hızlı yol, üstteki düğümün adresinin iki katının bir fazlasını sola, iki fazlasını sağa yazmaktır.

10.1 Kodla indis sütununu oluşturmak

Algoritma genişletilmiş diziyi alır; ağaç sondan kurulduğu için dizi sondan başa (-60'tan 2'ye) okunur. Kırmızıların ara düğüm, siyahların yaprak olduğunu bilgisayar eksi işaretinden anlar. Slayttaki kural (ekrandan, 02:20:00 ve 02:30:00):

> Her negatif sayı bir ara düğüm olduğu için bu düğümden çıkan dallar var demektir. Bu sebeple her negatif sayının sahip olduğu indis değerinin $\cdot 2 + 1$ ile $\cdot 2 + 2$ değeri peş peşe indis sütununa yazılır. Örneğin: -60 bir ara düğümdür. İndis değeri 0'dır. $\cdot 2 + 1$ ve $\cdot 2 + 2$ değerleri sırası ile 1 ve 2 olup arka arkaya yazılır.

Adım adım:

- -60: kök, indis 0. Negatif; $0 \cdot 2 + 1 = 1$ ve $0 \cdot 2 + 2 = 2$ yazılır. Gerçekten 1. gözde -35, 2. gözde 25 vardır.
- -35: indis 1. Negatif; 3 ve 4 yazılır (-21 ve 14).
- 25: pozitif, yaprak; indisiyle bir şey yapılmaz, altında dal yoktur.
- -21: indis 3. Negatif; 7 ve 8 yazılır (-13 ve 8).
- 14: pozitif, işlem yok.
- -13: indis 7. Negatif; 15 ve 16 yazılır (7 ve -6).
- 8 ve 7: pozitif, işlem yok.
- -6: indis 16. Negatif; 33 ve 34 yazılır (4 ve 2).
- 4 ve 2: pozitif, işlem yok.

Sonuç tablo (ekrandan, 02:30:00):

dizi	-60	-35	25	-21	14	-13	8	7	-6	4	2
indis	0	1	2	3	4	7	8	15	16	33	34

11. İndislerden Kod Üretmek

Kodlara yalnız yapraklar, yani pozitif (siyah) elemanlar için ihtiyaç vardır; bunlar birer sembole karşılık gelir. Slayttaki kural (ekrandan, 02:20:00):

- Huffman ağacı kodları aşağıdan yukarı doğru oluşturduğundan kodların prefix özelliğini sağlaması için de yukarıdan aşağıya doğru okunması gerekir.
- Matriste kod sütununun oluşturulması sırasında da ağaca indisleri yerleştirirken yaptığımız işlemin tersini yapmamız gerekir.
- Dizinin pozitif elemanları ağacın yapraklarına karşılık gelip sembolü temsil ettiğinden sahip olduğu indis değeri ile işlem yapılır, elde edilen kod değerleri de sağdan sola doğru yazılır.

Slayttaki sözde kod (ekrandan, 02:20:00; slayttaki karşılaştırma işareti hoca tarafından karalanmış, hoca koşulu "indis sıfırdan farklı" diye düzeltti):

```
While indis != 0 do
{
    if indis çift then
    {
        print "1"
        indis = (indis-2)/2
    }
    else
    {
        print "0"
        indis = (indis-1)/2
```

```
    }
}
```

Mantık: indis yerleştirirken sol çocuk için $\cdot 2 + 1$ (tek sayı), sağ çocuk için $\cdot 2 + 2$ (çift sayı) yapılıyordu. Şimdi ters işlemle yukarı çıkılır: tek indisten 1 çıkarılıp 2'ye bölünür (sol daldan gelinmiştir, etiket `0`), çift indisten 2 çıkarılıp 2'ye bölünür (sağ daldan gelinmiştir, etiket `1`). İndis 0 olunca köke varılmıştır, dur.

Hocanın çözdüğü örnekler:

- **25 (indis 2):** 2 çift → `1`; $(2-2)/2 = 0$; dur. Kod `1`.
- **14 (indis 4):** 4 çift → `1`; $(4-2)/2 = 1$. 1 tek → `0`; $(1-1)/2 = 0$; dur.
- **8 (indis 8):** 8 çift → `1`; $(8-2)/2 = 3$. 3 tek → `0`; $(3-1)/2 = 1$. 1 tek → `0`; 0; dur.
- **7 (indis 15):** 15 tek → `0`; 7. 7 tek → `0`; 3. 3 tek → `0`; 1. 1 tek → `0`; 0; dur.
- **4 (indis 33):** 33 tek → `0`; 16. 16 çift → `1`; 7. 7 tek → `0`; 3 → `0`; 1 → `0`; 0; dur.
- **2 (indis 34):** 34 çift → `1`; 16. 16 çift → `1`; 7 → `0`; 3 → `0`; 1 → `0`; 0; dur.

İlk anlatımda hoca bitleri üretildikleri sırayla soldan sağa yazdı (ekrandan, 02:27:45): `1`, `10`, `100`, `0000`, `01000`, `11000`. Bunlar prefix kod değildir: `1` ile başlayan ve `10` ile başlayan birden çok kod vardır. Nedeni, kodların aşağıdan yukarı (yapraktan köke) üretilmesidir; yukarıdan aşağı okunmaları gerekir. Bu yüzden ikinci bir adım olarak kodlar ters çevrilir: `1`, `01`, `001`, `0000`, `00010`, `00011`. Ya da bitler baştan sağdan sola yazılır ("Arapça gibi") ve çevirmeye gerek kalmaz. Hangisinin seçileceği kişiye kalmıştır. Slayttaki tablo (ekrandan, 02:20:00):

dizi	indis	kod
25	2	`1`
14	4	`01`
8	8	`001`
7	15	`0000`
4	33	`00010`
2	34	`00011`

Bütün kodlar python ile sözde koddan üretilip bu tabloyla karşılaştırıldı ve tuttu. Bunlar Bölüm 7.3'te ağaçtan okunan kodlarla aynıdır.

> **Ders sonrası not:** Hoca ikinci anlatımda (02:32:35 sonrası) kuralı "çift sayılarda 0, tek sayılarda 1 yazıyordum" diye söyledi ve ekrana sağdan sola `0`, `10`, `110`, …, `11100` yazdı (ekrandan, 02:34:40). Bunlar doğru kodların tümleyenidir (her bit ters). Slayttaki sözde koda ve ağaçtaki etiketlere göre çift indis (sağ çocuk, $2i + 2$) için `1`, tek indis (sol çocuk, $2i + 1$) için `0` yazılır. Ayrıca hoca 02:24:38'de ilk anlatımda da 34 için bir kez "0" dedi; doğrusu 1'dir.
>
> **Ders sonrası not:** Hoca 02:28:27'deki özet tekrarında diziyi "iki en eksi iki" olacak şekilde genişlettiğini söyledi; dersin örneklerinde kullanılan ve slaytta yazan kural $2n - 1$'dir.

11.1 Bütün süreç kod olarak

Hoca süreci bir kez daha toparladı. Elle yapılan her adımın koddaki karşılığı vardır:

- Frekanslar herhangi bir sıralama algoritmasıyla küçükten büyüğe sıralanır.
- İkişer toplanıp toplamlar sıralamayı bozmadan araya yerleştirilerek dizi $2n - 1$ elemana genişletilir; toplamlar eksi işaretle tutulur.
- Dizi sondan okunarak indis sütunu oluşturulur: negatif elemanların indisinin $2i + 1$ ve $2i + 2$ değerleri peş peşe yazılır.
- Pozitif elemanların (yaprakların) indislerinden ters işlemle kodlar üretilir ve sağdan sola yazılır.
- Ağaç bilgisi header'a gönderilecekse ağaç left-node-right sırasıyla okunur.

> **Hocanın vurgusu:** Huffman ağacını kod yazarak elde etmek "çok kolay"; çizimde gösterilen her şeyin karşılığı bu dizi işlemleridir.

12. Sembol Türleri

Sembollerin her zaman tek karakter olması gerekmez. İkili karakterler kodlanabilir: `ab`'ye, `ek`'e, `ke`'ye, `la`'ya birer kod verilebilir. Üçlü karakterler, hatta kelimeler de ("gel", "yaz" gibi) kodlanabilir. İster bir karakter ister n karakter olsun, işlem aynıdır. Yalnız header'da sembollerin ne olduğu söylenmelidir; örneğin "kelimelerim en fazla 8 harften oluşuyor, hep 8'er bayt oku" denir. Önceki örnekte karakterler kodlandığı için bayt bayt okunmuştu.

13. Statik, Yarı Dinamik ve Dinamik Huffman

Hoca dinamik Huffman'ın daha karışık olduğunu, ayrıntısına sonraki derste geçeceğini söyledi; bu derste üç türün tanımlarını verdi (ekrandan, 02:40:00). Huffman algoritmaları üç başlıkta incelenir.

13.1 Statik Huffman

Slayttaki tanım: bir derlem içerisinde yer alan tüm sembollerin kullanım sıklıkları çıkarıldıktan sonra her sembol için Huffman kod değerleri oluşturulur. Gelen her doküman için aynı kod değerleri kullanılır; doküman içindeki sembollerin kullanım sıklıkları dikkate alınmaz.

Hoca bunu ASCII tablosuna benzetti: ASCII'yi birileri oturup hazırlamıştır; dünyanın neresinde olunursa olunsun `a`'nın kodu değişmez. Statik Huffman'da da örneğin Wikipedia'daki bütün İngilizce sayfalar bir derlem (korpus) olarak alınır, karakterlerin olasılıkları çıkarılır, Huffman kodları oluşturulur ve "hangi İngilizce metin gelirse gelsin bu kodlarla kodlayacaksın" denir. Türkçe için de Wikipedia'daki Türkçe sayfalardan aynısı yapılır. Bu çok basit ve kullanışlıdır, ama iyi bir performans vermeyebilir. Dokümanda geçmeyen sembollerin bile kodu vardır; o semboller olmasa Huffman ağacının boyu kısalır, semboller daha az bitle kodlanır. Statik kodla gereksiz yere uzun kodlar kullanılmış olabilir.

13.2 Yarı dinamik Huffman

Slayttaki tanım: kodlanacak her doküman üzerinde iki aşamadan oluşan bir işlem yapılır. İlk olarak doküman baştan sona taranarak sembollerin kullanım sıklıkları çıkarılır. İkinci adımda da Huffman kod değerleri elde edilir.

Derste gösterilen yöntem budur: sembollerin olasılıkları ve kodları önceden bilinmez, o an gelen dokümana göre hazırlanır. Hocanın örneğinde bir doküman Ayşe'ye gelir; Ayşe dokümandaki karakterlerin sıklıklarını bulur, kodları çıkarır ve kendi dokümanına göre örneğin `a`'ya `0`, `e`'ye `11` verir. Ali'nin dokümanında da `a` ve `e` vardır ama sayıları farklıdır; orada `a`'ya `11`, `e`'ye `101` gibi kodlar verilebilir. Birinci aşamada doküman okunur, frekanslar çıkarılır, olasılıklar hesaplanır, ağaç oluşturulur ve kodlar çıkarılır; ikinci aşamada doküman yeniden baştan, bu kez kodlamak (sıkıştırmak) için okunur.

Statik Huffman yarı dinamikten daha hızlı çalışır, ama performansı farklıdır. Hoca örnek olarak statik ile %50 gibi bir sıkıştırma elde ediliyorsa yarı dinamik ile %65'lerin elde edilebileceğini söyledi.

13.3 Dinamik Huffman

Slayttaki tanım: hızın önemli olduğu uygulamalarda kullanılır; encoder ve decoder senkronize çalışarak aynı anda kodlama ağacını oluşturur (ekrandan, 02:40:00). Hoca şöyle açıkladı: statik ideal kodları vermez, yarı dinamik metni iki kez okur; dinamik Huffman'da böyle bir şey yoktur. Metni okuduğu an kodu oluşturur ve karşıya gönderir; karşı taraf da aynı anda kendi ağacını oluşturur ve iki taraftaki ağaç aynıdır. Bir sembolün kodu dokümanın başında, ortasında ve sonunda farklı olabilir; örneğin `a`'ya önce 2 bitlik, sonra 5 bitlik, sonra 3 bitlik kodlar verilebilir.

Tek problem, varyansı küçük bir ağaç gerekmesidir; yoksa decode ederken beklemeler oluşur, buffer alanı açmak gerekir ve gereksiz bellek kullanılır. Bu yüzden en ideali varyansı düşük bir kod bulmaktır.

Slaytta dinamik Huffman için ayrıca başlangıçta ağacın boş olduğu, ilk sembolün kodlanmadan gönderildiği, kodlanmamış sembolden önce bir escape kodu gönderildiği gibi maddeler yer alır (ekrandan, 02:40:00); hoca bunların ayrıntısını sonraki derse bıraktı.

Örnek Sorular

1. İki ağaçtan hangisi daha iyidir? (ekrandan, 00:40:00)

Olasılıkları 0,4; 0,2; 0,2; 0,1; 0,1 olan beş sembol için iki Huffman kodu elde edilmiştir: birinci kod `0, 10, 111, 1101, 1100`, ikinci kod `11, 01, 00, 101, 100`. Hangi kod daha iyidir?

> **Çözüm (derste verildi):** İki kodun ortalama uzunluğu da $S = 2{,}2$ bit/sembol, entropileri ve redundancy'leri aynıdır. Varyanslar: birinci kod $1{,}36$, ikinci kod $0{,}16$. Veri dosyaya yazılacaksa fark etmez; iletişim hattından gönderilecekse küçük varyanslı ikinci kod seçilir, çünkü daha küçük bir buffer yeterlidir.

2. Frekanslarla Huffman kodu (ekrandan, 00:50:00)

Elimizde a, e, k, l, m ve z sembollerinden sırası ile 25, 14, 8, 7, 4 ve 2 adet olsun. Her bir sembol için Huffman kod değerini bulalım.

> **Çözüm (derste verildi):** Sıralı dizi `2 4 7 8 14 25`; genişletilmiş dizi `2 4 -6 7 8 -13 14 -21 25 -35 -60` ($2n-1 = 11$ eleman). Kök -60; çocuklar (-35, 25), (-21, 14), (-13, 8), (7, -6), (4, 2). Sol 0, sağ 1; kökten yaprağa: `a = 1`, `e = 01`, `k = 001`, `l = 0000`, `m = 00010`, `z = 00011`.
>
> **Çözüm (derste verilmedi):** Ortalama kod uzunluğu python ile $S = (25\cdot 1 + 14\cdot 2 + 8\cdot 3 + 7\cdot 4 + 4\cdot 5 + 2\cdot 5)/60 = 135/60 = 2{,}25$ bit/sembol; ara düğümlerin toplamı da $(60+35+21+13+6)/60 = 2{,}25$ verir. Entropi $H \approx 2{,}19$ bit, redundancy $R \approx 0{,}06$ bittir.

3. Sınıf alıştırması (ekrandan, 01:10:00; derste sözlü, 01:06:34)

Sembollerin frekans değerleri 1, 1, 1, 2, 3, 5, 6 ve 11 olarak verilsin. Huffman ağacını oluşturun.

> **Çözüm (derste verildi):** $n = 8$, genişletilmiş dizi 15 elemanlı. Ara toplamlar sırasıyla 2, 3, 5, 8, 11, 19, 30; kök 30. Kodlar: 11 → `00`, 5 → `010`, 2 → `0110`, 1 → `0111`, 6 → `10`, 3 → `110`, 1 → `1110`, 1 → `1111` (ekrandan, 01:17:45). Ortalama kod uzunluğu (derste hesaplanmadı, python ile) $78/30 = 2{,}6$ bit/sembol.

4. Header'da ne gönderilmeli? (derste sözlü, 01:40:31)

A'dan 175, B'den 512, C'den 1 tane olan bir veri, diyelim ki C = `0`, B = `10`, A = `110` kodlarıyla sıkıştırılıp karşıya gönderildi. Karşı tarafın bunu açabilmesi için nelere ihtiyacı vardır?

> **Çözüm (derste verildi):** Yalnız semboller ve kodlar gönderilirse değişken uzunluklu kodlar ayrılamaz; kod uzunlukları ya da frekanslar eklenirse header büyür ve alıcı ağacı baştan kurmak zorunda kalır. En küçük header, Huffman ağacının kendisini göndermektir: sembol sayısı, yaprak sırasıyla semboller ve ağacın left-node-right dolaşımındaki dal etiketleri.

5. Başlık içerisinde ağaç bilgisini gönderebilir miyiz? Nasıl? (ekrandan, 01:50:00 ve 02:00:00)

Bölüm 7'deki ağaç için header'ı oluşturun ve kaç bayt tuttuğunu bulun.

> **Çözüm (derste verildi):** Ağaç Left Node Right sırasıyla okunur: dal etiketleri `0000101111`, yapraklardaki semboller `l m z k e a`. Header: 1 bayt sembol sayısı + 6 bayt sembol + 10 bit için 2 bayt (son bayt sıfırla doldurulur) = 9 bayt.

6. Genişletilmiş diziyi kullanarak kodları nasıl oluşturabiliriz? (ekrandan, 02:10:00 ve 02:20:00)

Genişletilmiş dizi `2 4 -6 7 8 -13 14 -21 25 -35 -60` için indis sütununu ve kod sütununu doldurun.

Çözüm (derste verildi): Dizi sondan okunur; kökün indisi 0; her negatif elemanın indisinin $2i+1$ ve $2i+2$ değerleri peş peşe yazılır: -60 → 0, -35 → 1, 25 → 2, -21 → 3, 14 → 4, -13 → 7, 8 → 8, 7 → 15, -6 → 16, 4 → 33, 2 → 34. Pozitif elemanların indisinden, indis çiftse `1` yazıp $(i-2)/2$, tekse `0` yazıp $(i-1)/2$ alınarak indis 0 olana kadar devam edilir; bitler sağdan sola yazılır: 25 → `1`, 14 → `01`, 8 → `001`, 7 → `0000`, 4 → `00010`, 2 → `00011`.

Dersin Toparlaması

Bu derste Huffman kodlaması baştan sona işlendi. Huffman, kodları yapraklardan köke doğru oluşturur: en küçük iki değer toplanır, toplam sıralamayı bozmadan diziye yerleştirilir ve bu işlem tek bir değer (kök) kalana kadar sürer. Eşit değerlerde toplamın önüne mi arkasına mı konacağı serbesttir, ama seçilen kural hep aynı uygulanmalıdır. Bu seçim farklı ağaçlar ve farklı kodlar verir; ortalama kod uzunluğu ve entropi değişmez, ama varyans değişir. Dosyaya yazarken varyans önemsizdir, iletimde ise küçük varyanslı kod daha küçük buffer ile çalışır. Kodlar kökten yaprağa okunmalıdır; ters okunursa prefix özelliği bozulur. Ortalama kod uzunluğu ara düğümlerin toplamıyla da bulunabilir.

Uygulama tarafında frekanslarla çalışan bir yöntem gösterildi: genişletilmiş $2n-1$ elemanlı dizide ara toplamlar eksi işaretle tutulur, dizi sondan okunarak ağaç kurulur. Karşı tarafın veriyi açabilmesi için en küçük header ağacın kendisidir: left-node-right dolaşımındaki dal etiketleri ve yaprak sırasıyla semboller (örnekte 9 bayt). Kod yazarken ağaç, sol çocuk $2i+1$, sağ çocuk $2i+2$ adreslemesiyle bir diziye aktarılır ve yaprakların indislerinden ters işlemle kodlar üretilir. Son olarak statik, yarı dinamik ve dinamik Huffman ayrımı yapıldı. Hoca sonraki derste dinamik Huffman ve canonical Huffman'a, ardından MNP5, MNP7 gibi Microcom ağ protokollerine ve faks sıkıştırmasına geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.